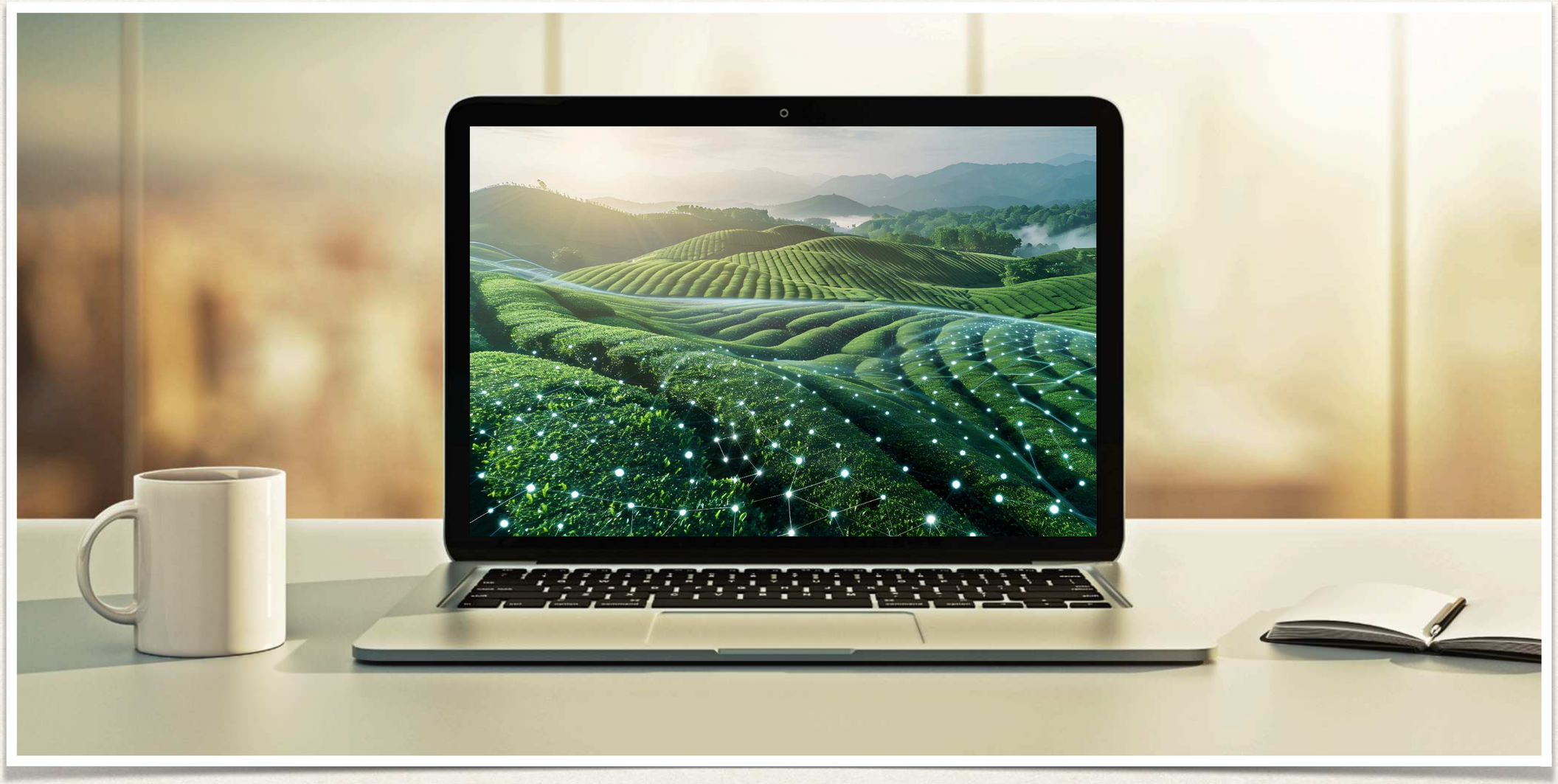
Books Project



What will we be creating?

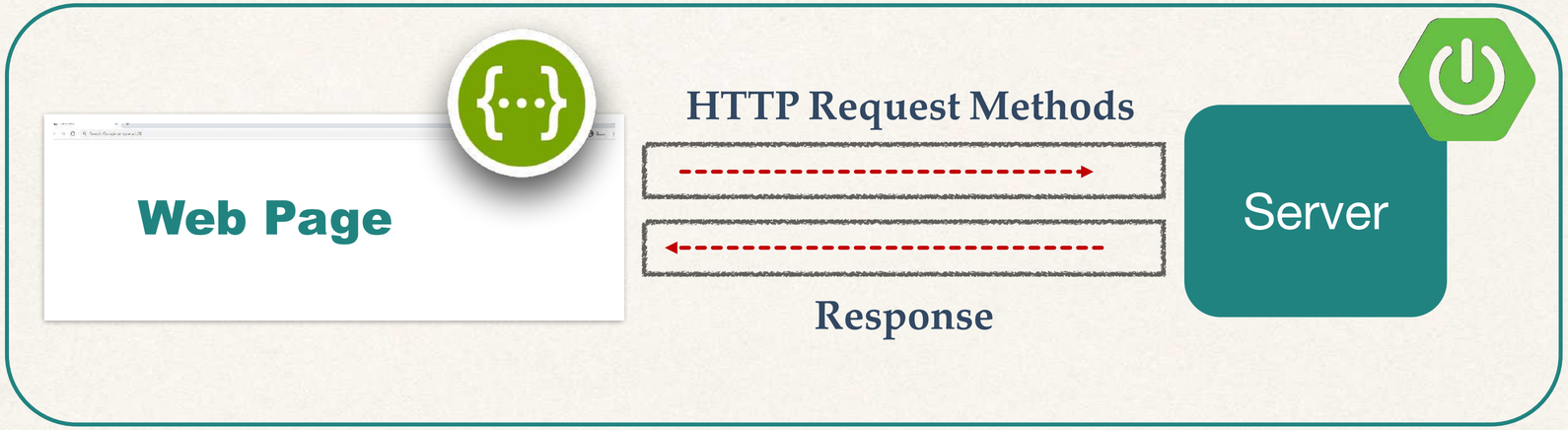
- Creating and enhancing books to learn the basics of Spring Boot Rest

```
public void initializeBooks() {
    books.addAll(List.of(
      new Book("Title One", "Author One", "science"),
      new Book("Title Two", "Author Two", "science"),
      new Book("Title Three", "Author Three", "history"),
      new Book("Title Four", "Author Four", "math"),
      new Book("Title Five", "Author Five", "math"),
      new Book("Title Six", "Author Two", "math")
    ));
}
```

CRUD Operations

Create | Read | Update | Delete

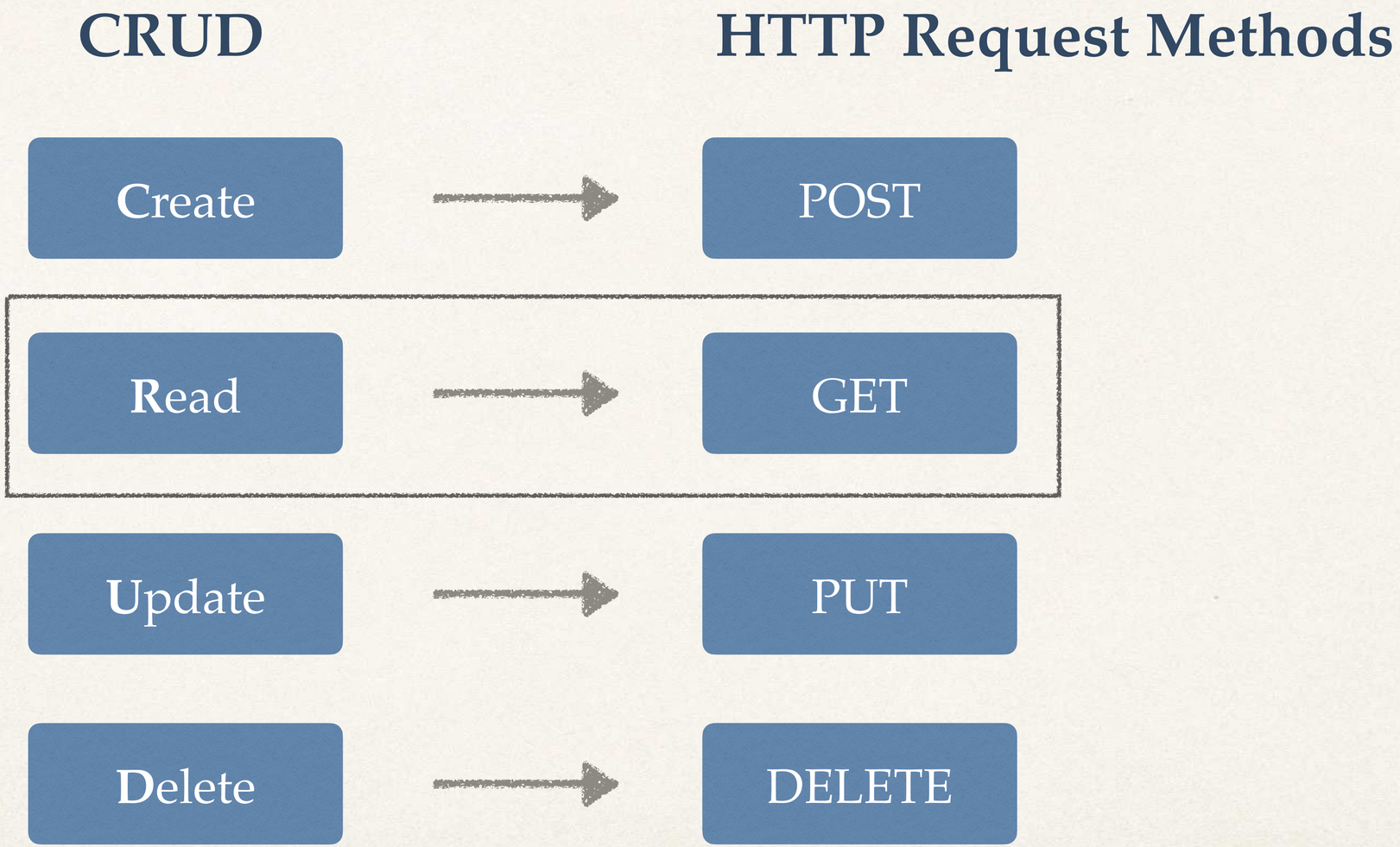
Request and Response



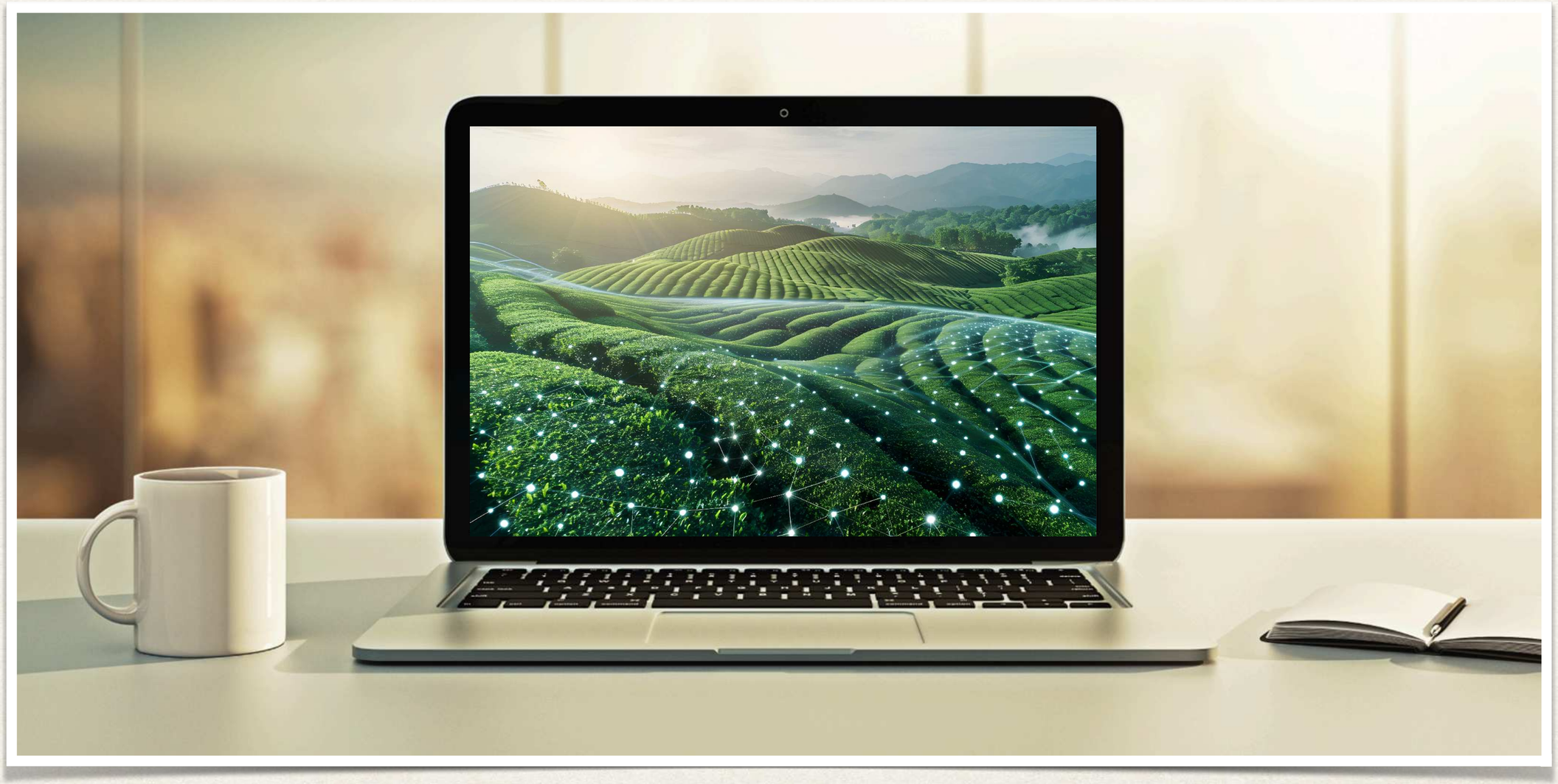
CRUD Operations

- Create
- Read
- Update
- Delete

HTTP Request Methods



Setup our Spring Boot Project



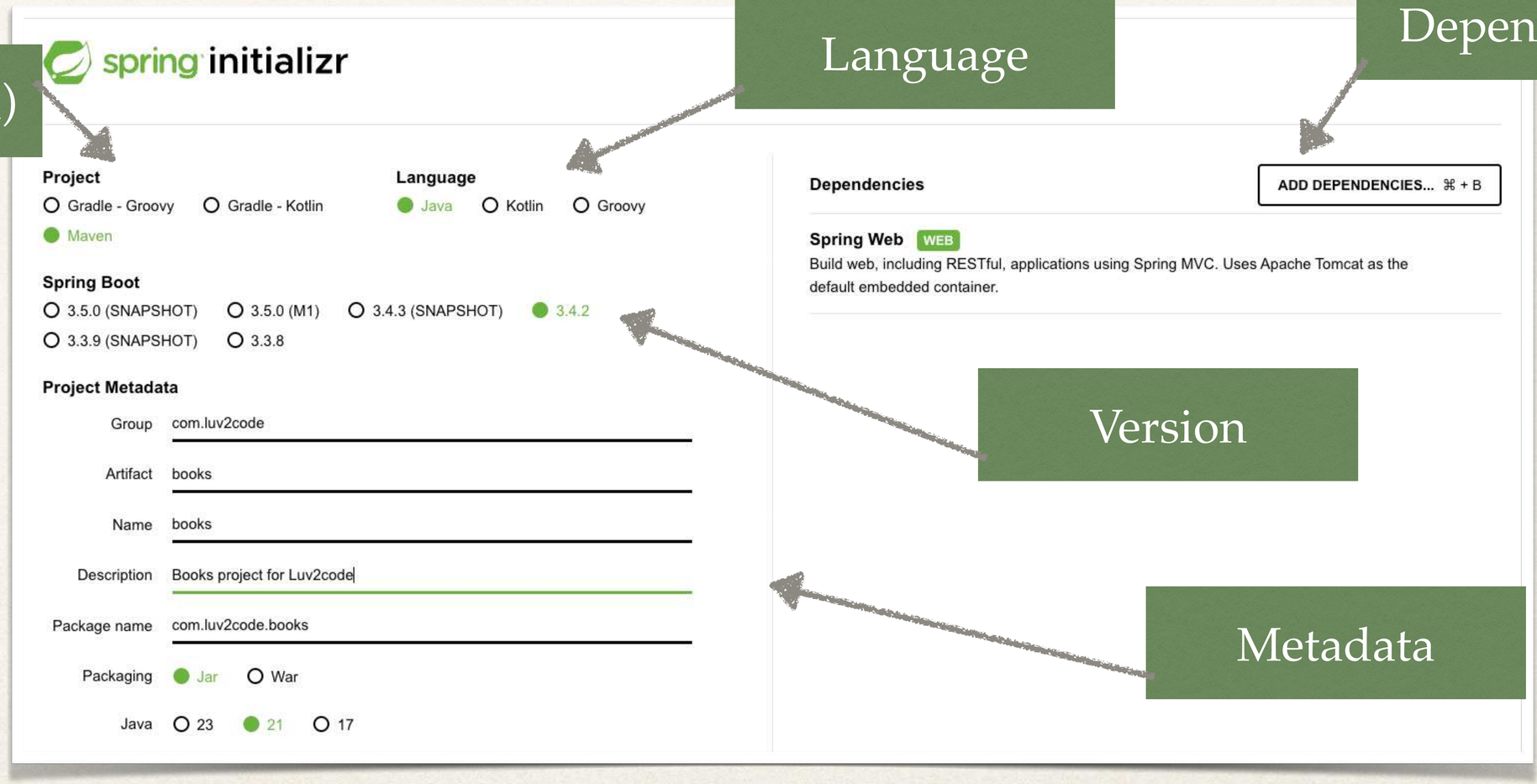
Spring Initializr

- Here we can set the project configurations for our upcoming Spring project.
- Allows us to choose the project, programming language, Spring Boot version, dependencies and metadata for our application.

start.spring.io

Spring Initializr

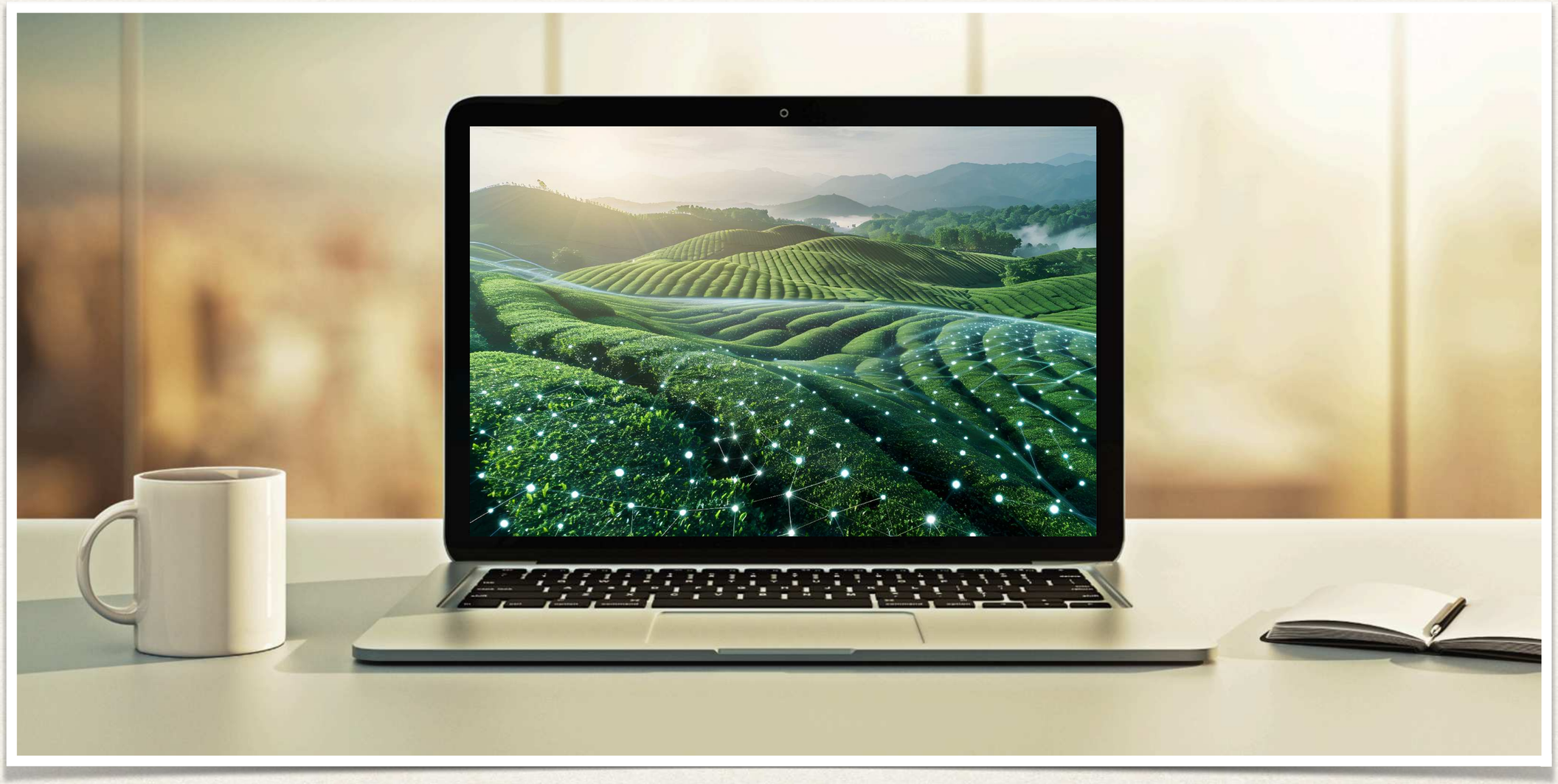
- Setup our project using Spring Initializr



Spring Initializr

- Spring Initializr will download a .zip file onto our machine.
- We will need to unzip the .zip file.
- Open the project on IntelliJ.
- Download all dependencies using Maven.
- Sounds fun - Let's go ahead and dive in!

GET HTTP Request Method



Creating a Spring Boot Rest endpoint (updated)

File: BookController.java

```
import org.springframework.web.bind.annotation.*;


@RestController
public class Controller {

@GetMapping(“/api-endpoint”)
public String firstApi() {
    return “Hello Eric!"
   }
}
```

Lets dive into the first function

Dive in

Read

Get

API Endpoint:

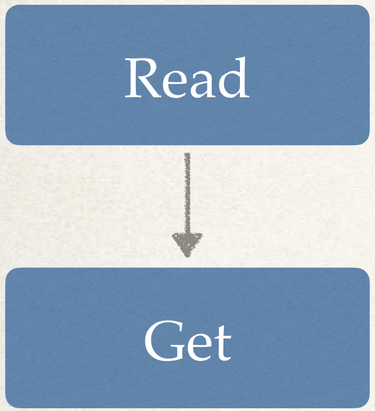
```
@GetMapping(“/api-endpoint”)
public String firstApi() {
    return “Hello Eric!"
}
```

URL : localhost:8080/api-endpoint

Response:

```
{
    “Hello Eric!”
}
```

Start Spring Boot Rest Application



API Endpoint:

```
@GetMapping("/api-endpoint")
public String firstApi() {
    return "Hello Eric!"
}
```

Run Spring Boot application:

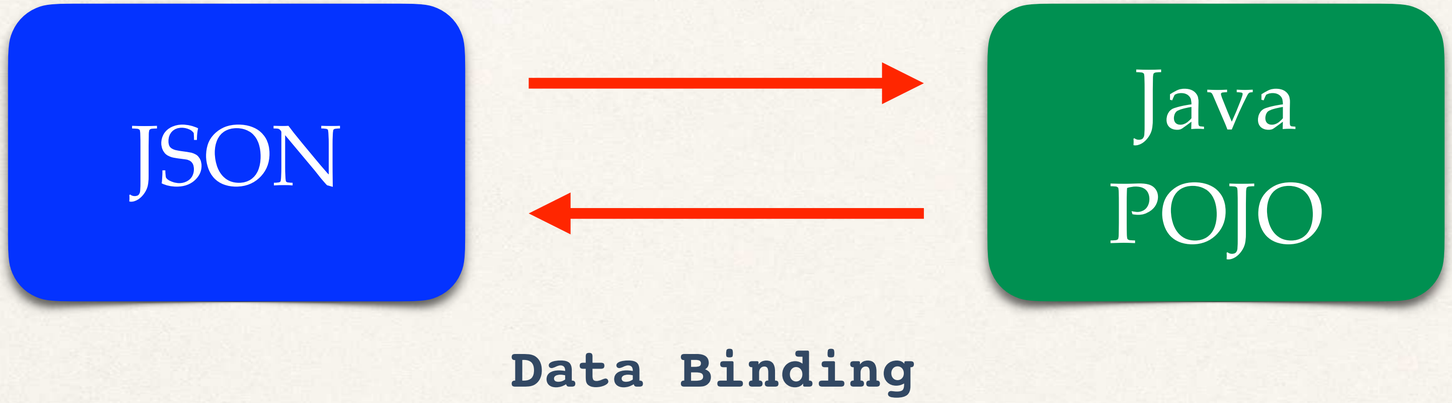
```
ericroby@Erics-MacBook-Pro ~% ./mvnw spring-boot:run
```

URL : localhost:8080

Java JSON Data Binding

Java JSON Data Binding

- Data binding is the process of converting JSON data to a Java POJO



Also known as

Mapping

Serialization / Deserialization

Marshalling / Unmarshalling

JSON Data Binding with Jackson

- Spring uses the **Jackson Project** behind the scenes
- Jackson handles data binding between JSON and Java POJO
- Details on Jackson Project:

https://github.com/FasterXML/jackson-databind

Jackson Data Binding

- By default, Jackson will call appropriate getter/setter method

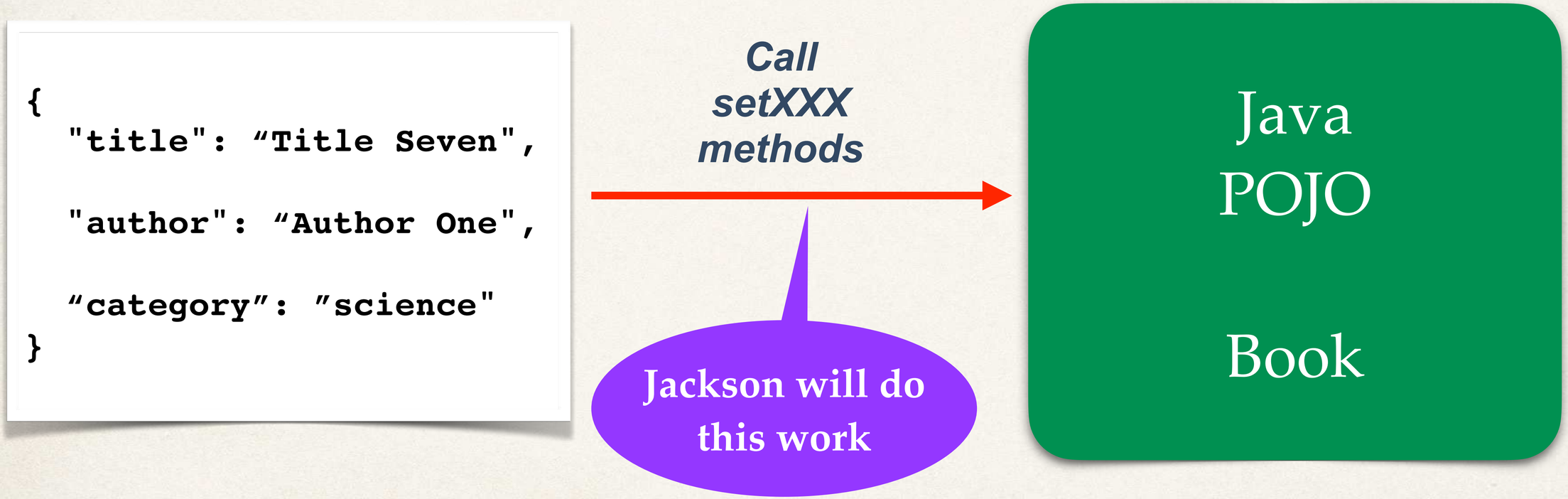
```
{
  "title": “Title Seven",

  "author": “Author One",

  “category”: ”science"
}
```

Java
POJO

Book

JSON to Java POJO

- Convert JSON to Java POJO … call setter methods on POJO

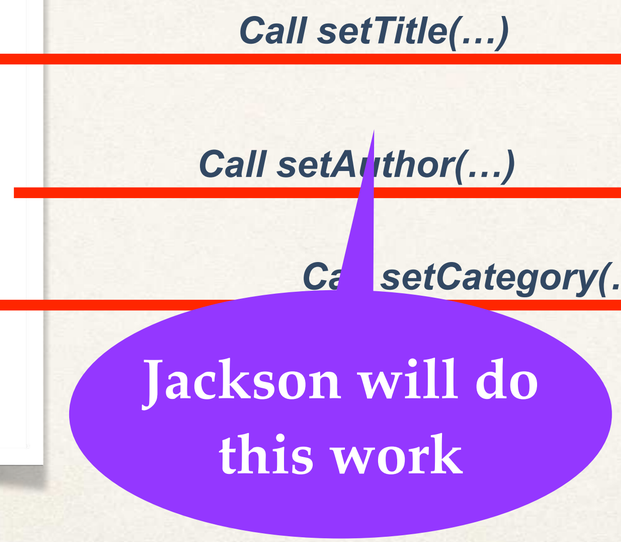


JSON to Java POJO

Note: Jackson calls the setXXX methods
It does NOT access internal private fields directly

- Convert JSON to Java POJO … call setter methods on POJO

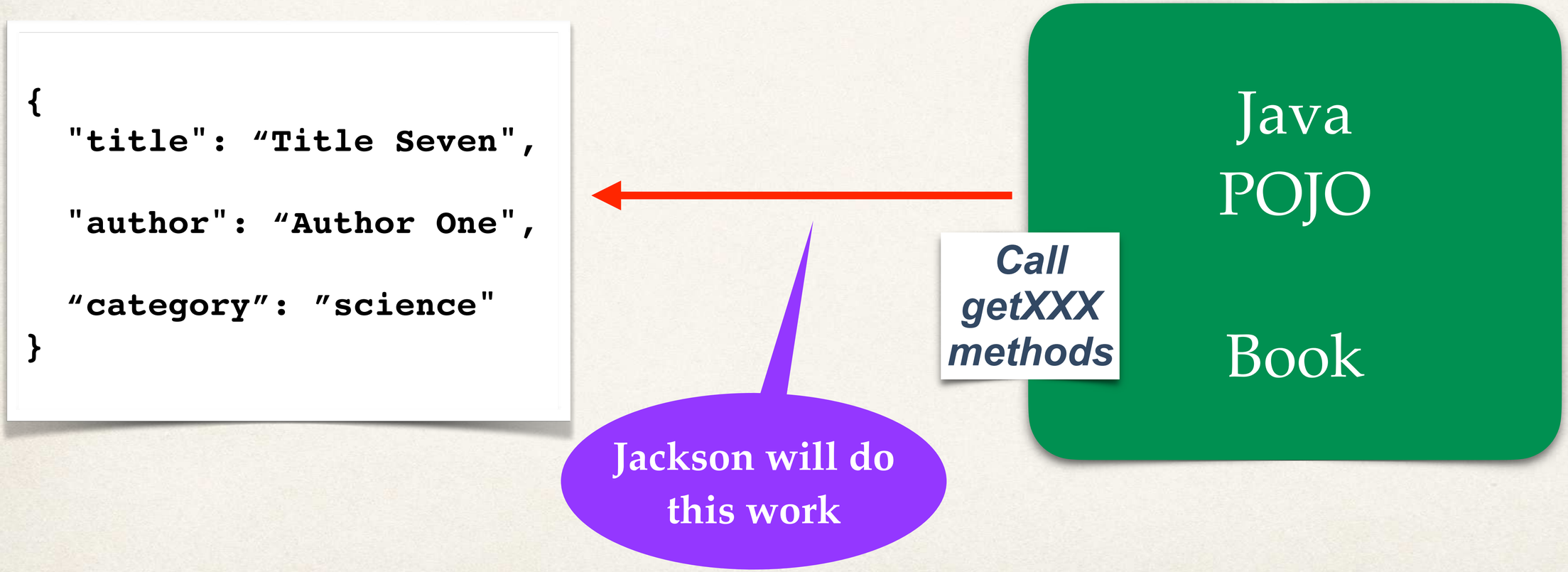
```
{
   "title": “Title Seven",

   "author": “Author One",

   “category”: ”science"
}
```




```
public class Book {

    private String title;
    private String author;
    private String category;

    public void setTitle(String title) {
        this.title = title;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public void setCategory(string category) {
        this.category = category;
    }

    // getter methods
}
```

Java POJO to JSON

- Now, let's go the other direction
- Convert Java POJO to JSON … call getter methods on POJO

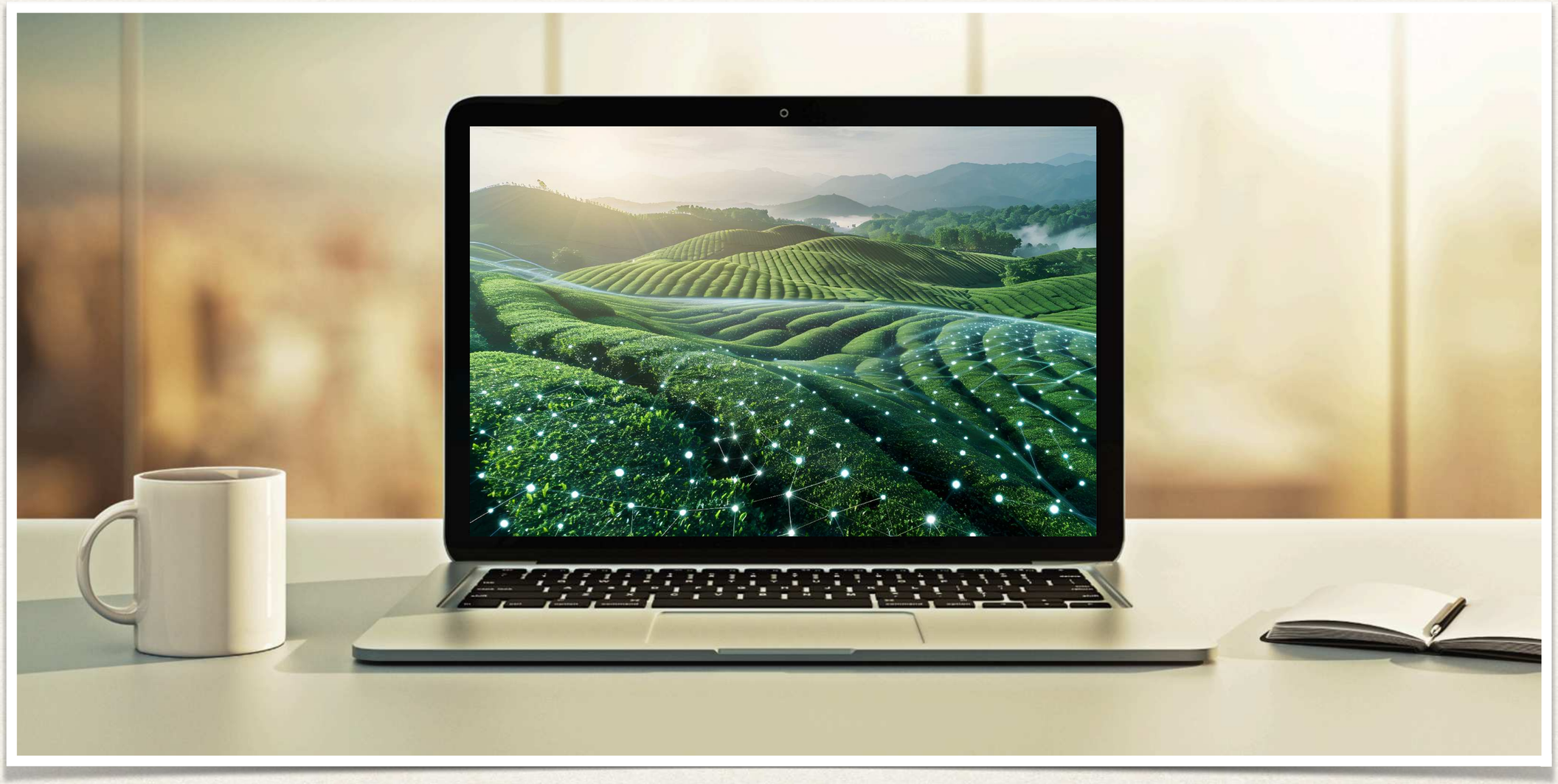


Spring and Jackson Support

- When building Spring REST applications
- Spring will automatically handle Jackson Integration
- JSON data being passed to REST controller is converted to POJO
- Java object being returned from REST controller is converted to JSON

Happens automatically behind the scenes

Spring REST Service - Books



Create a New Service

- Return a list of books

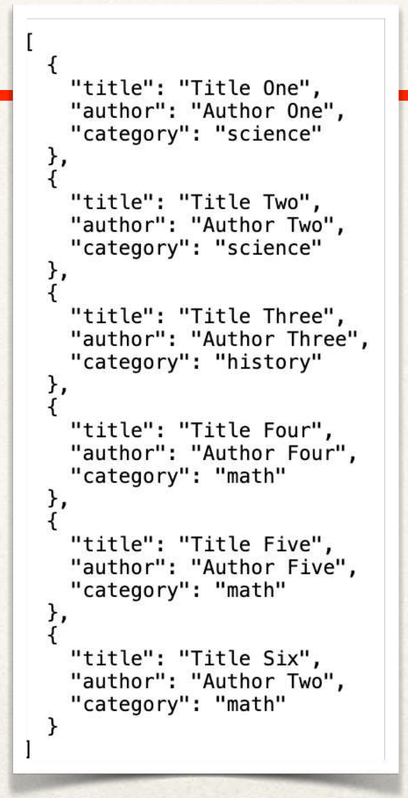
GET `/api/books` *Returns a list of books*

Spring REST Service

We will write this code

REST Client

/api/books



```
[
  {
    "title": "Title One",
    "author": "Author One",
    "category": "science"
  },
  {
    "title": "Title Two",
    "author": "Author Two",
    "category": "science"
  },
  {
    "title": "Title Three",
    "author": "Author Three",
    "category": "history"
  },
  {
    "title": "Title Four",
    "author": "Author Four",
    "category": "math"
  },
  {
    "title": "Title Five",
    "author": "Author Five",
    "category": "math"
  },
  {
    "title": "Title Six",
    "author": "Author Two",
    "category": "math"
  }
]
```

REST Service

Web Browser, Swagger Or Postman

Convert Java POJO to JSON

- Our REST Service will return **`List<Book>`**

- Need to convert **`List<Book>`** to JSON

- **Jackson** can help us out with this …

Spring Boot and Jackson Support

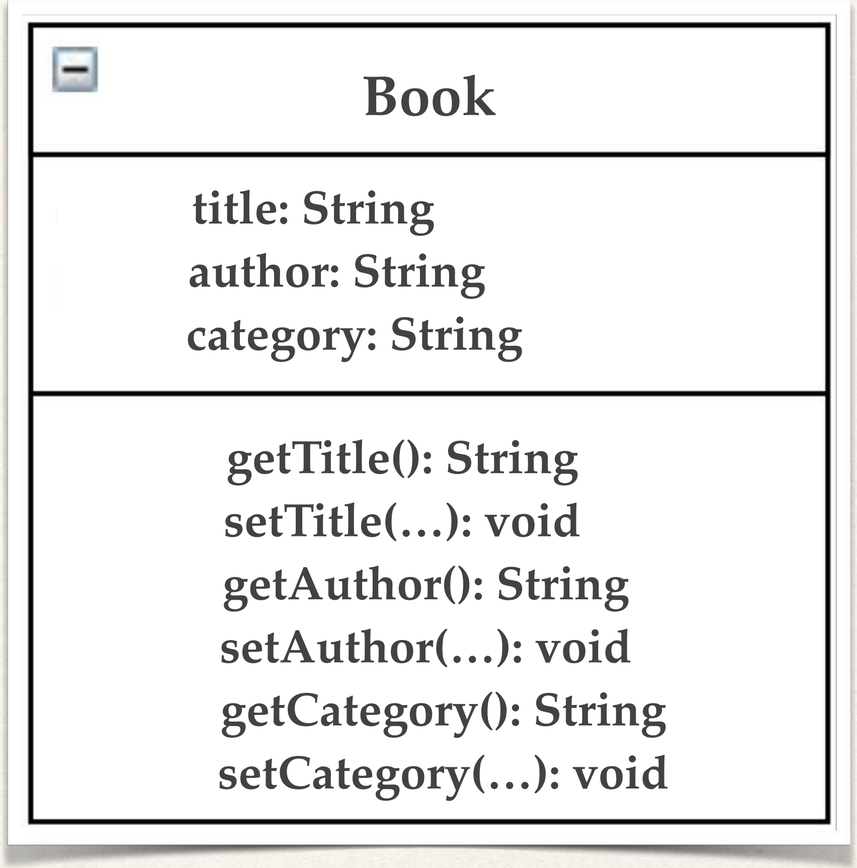
Happens automatically behind the scenes

- Spring Boot will automatically handle **Jackson** integration
- JSON data being passed to REST controller is converted to Java POJO
- Java POJO being returned from REST controller is converted to JSON

Book POJO (class)

Java
POJO

Book

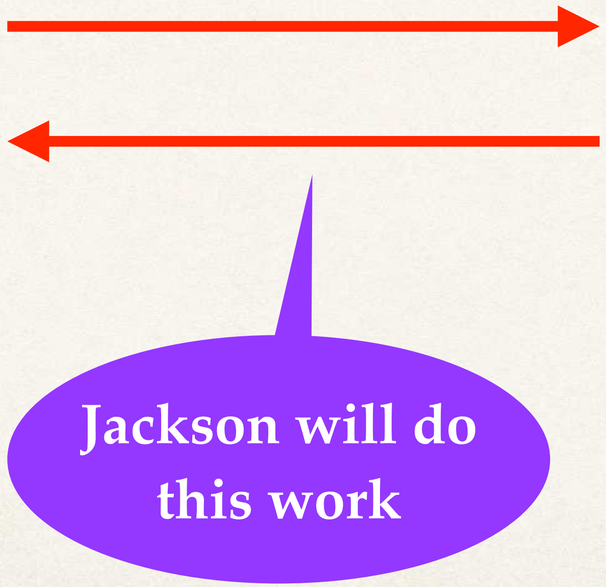


Jackson Data Binding

- Jackson will call appropriate getter / setter method

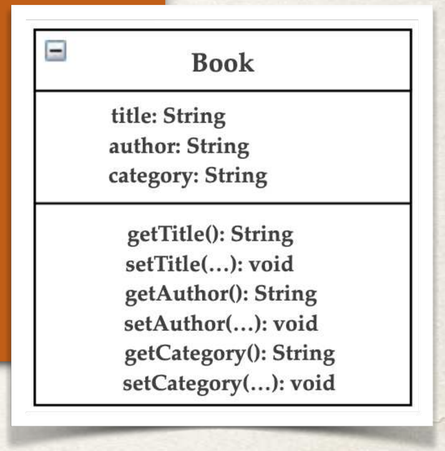
Remember

```
{
  "title": “Title Seven",

  "author": “Author One",

  “category”: ”science"
}
```



Java POJO

Book



Spring REST Service

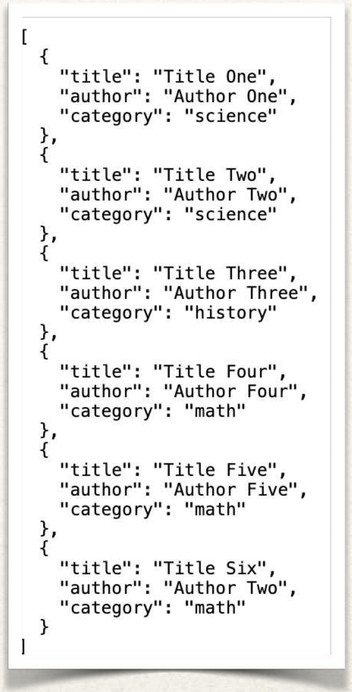
We will write this code

REST Client

/api/books

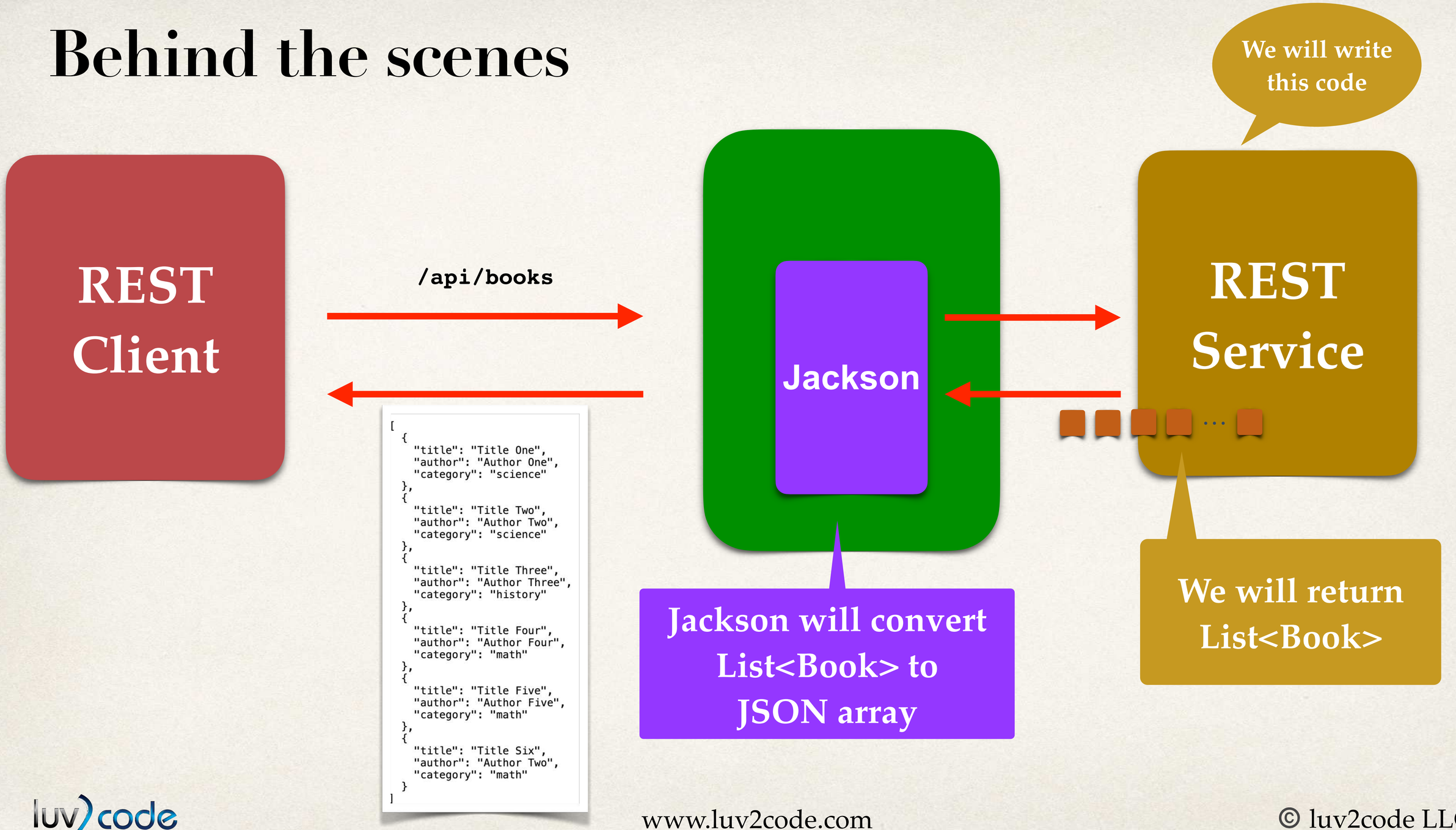
REST Service

List<Book>



Jackson will convert to JSON array

Behind the scenes



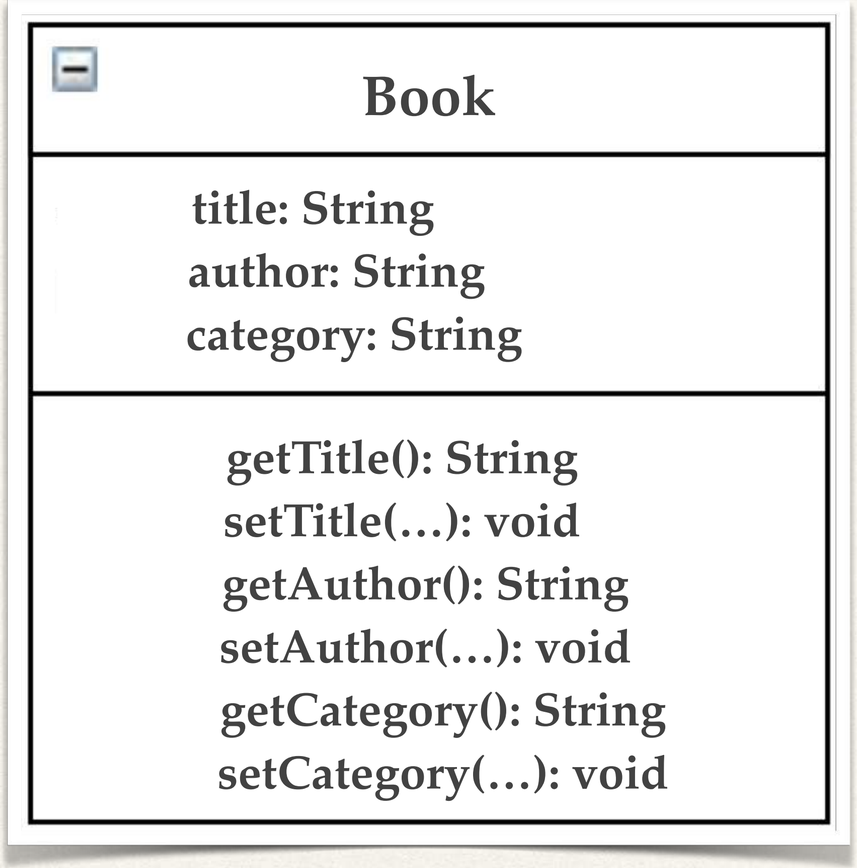
Development Process

Step-By-Step

1. Create Java POJO class for **Book**

2.Create Spring REST Service using **@RestController**

Step 1: Create Java POJO class for Book



File: Book.java

```
public class Book {

    private String title;
    private String author;
    private String category;

    public Book (String title, String author, string category) {
        this.title = title;
        this.author = author;
        this.category = category;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public void setCategory(string category) {
        this.category = category;
    }

    public String getTitle() {
        return title;
    }

   < Other Getters >
}
```

Fields
Constructors
Getter/Setters

Step 2: Create @RestController

File: BookController.java

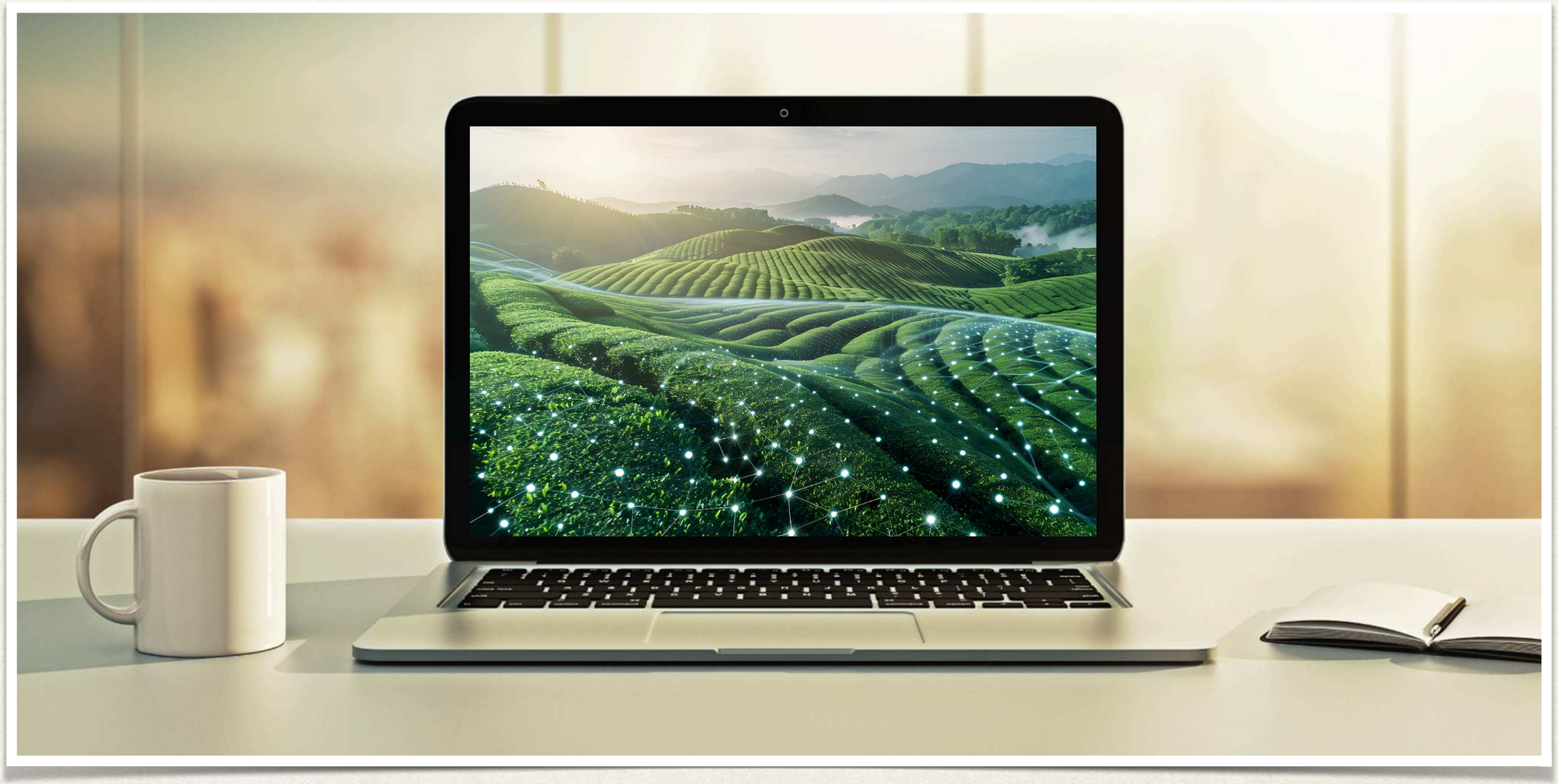
```
@RestController
public class BookController {

   // define endpoint for “/books" - return list of books

   @GetMapping(“/api/books")
   public List<Book> getBooks() {
      return books;
   }

}
```

```
public void initializeBooks() {
    books.addAll(List.of(
      new Book(“Title One”, “Author One”, “science”),
      new Book(“Title Two”, “Author Two”, “science”),
      new Book(“Title Three”, “Author Three”, “history”),
      new Book(“Title Four”, “Author Four”, “math”),
      new Book(“Title Five”, “Author Five”, “math”),
      new Book(“Title Six”, “Author Two”, “math”)
    ));
}
```

We'll hard code
for now …
can add DB later …

Jackson will convert
List<Book> to
JSON array

Swagger Setup



What is Swagger?

- Swagger is a set of tools and specifications for designing, building, documenting and consuming RESTful web services.
- Open-source project for streamlining API development by providing a common framework and API descriptions.
- Swagger UI implementation will allow us to be able to document and call our API endpoints right from our application.
- No third party tools will be needed!
- Win-win solution for implement Swagger within our application.

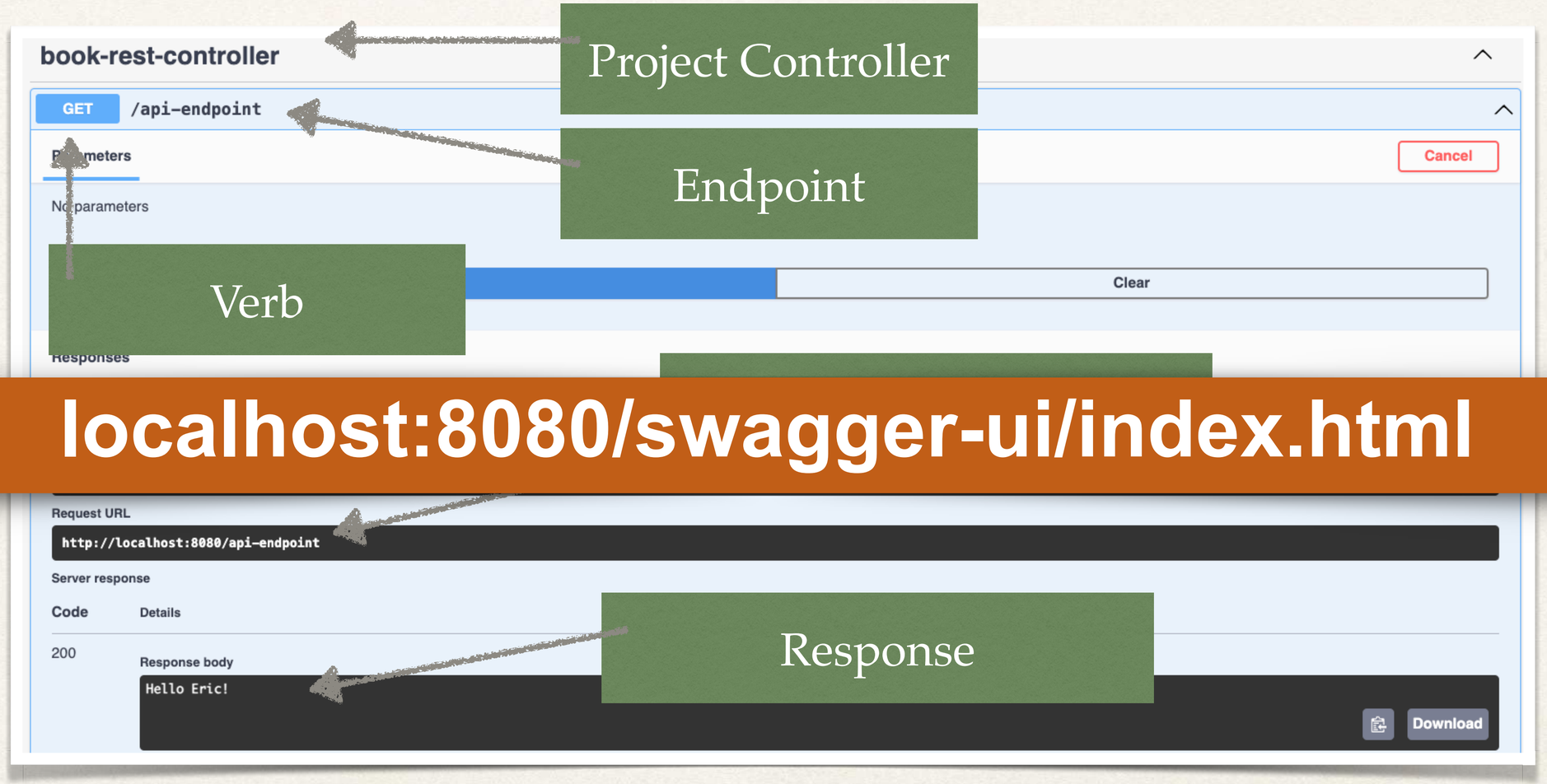
What is Swagger?

- Right now we go straight to the endpoint:

- However, once our application gets more complex, going straight to the endpoint will not be a good solution.
- This is due to security, authentication, HTTP request body/headers and more.
- We will cover the above later on, but for now, we need Swagger.

What is Swagger?

- Instead of going straight to the API endpoint, we can go to Swaggers.



What is Swagger?

- Path is hard to remember:
  - localhost:8080/swagger-ui/index.html

Too long to remember

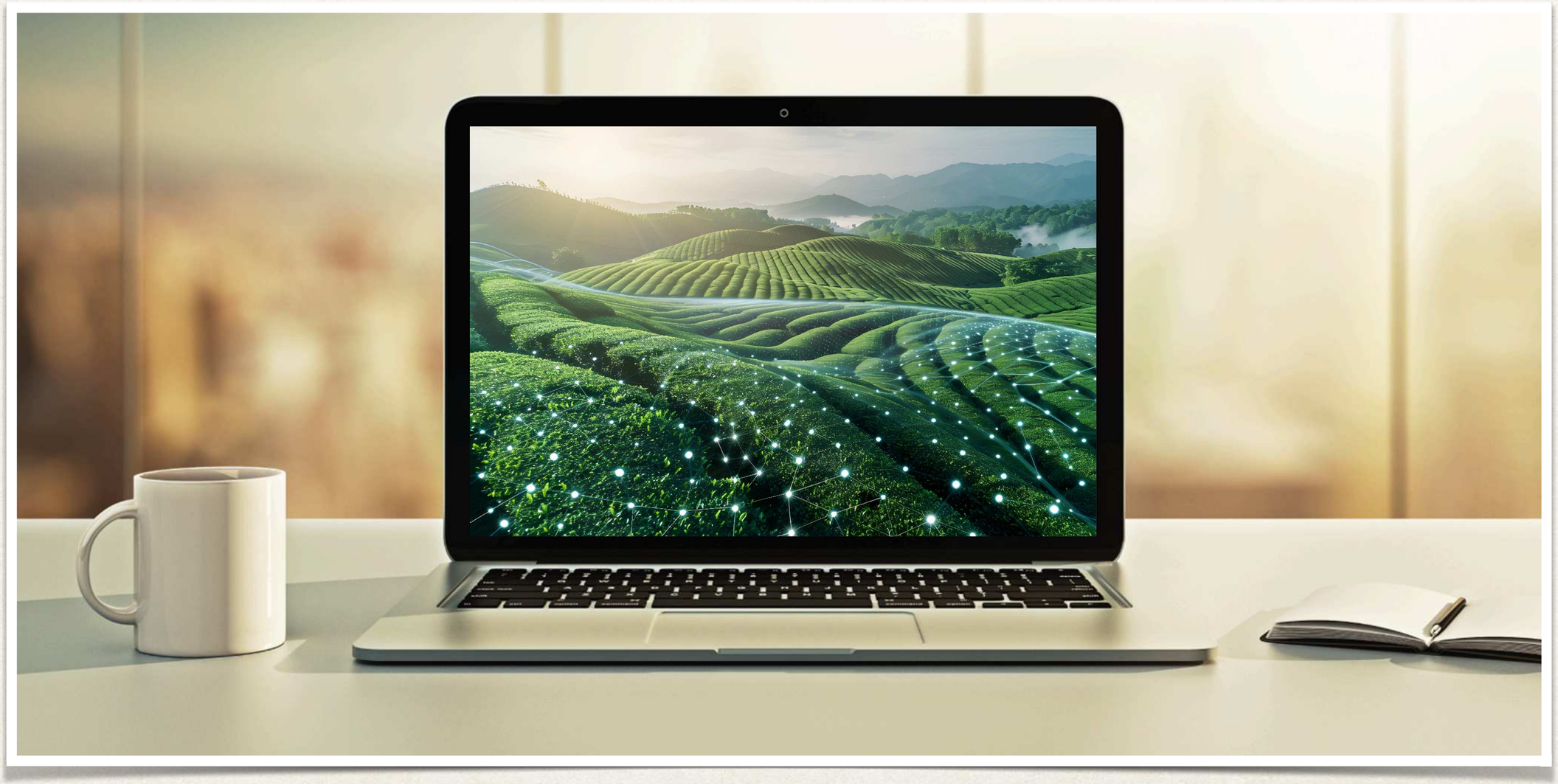
- We can fix this to redirect based off of /docs.
  - New endpoint for Swagger: localhost:8080/docs

Much better :-)

File: application.properties

```
springdoc.swagger-ui.path=/docs
```

Path Variables



What are Path Variables

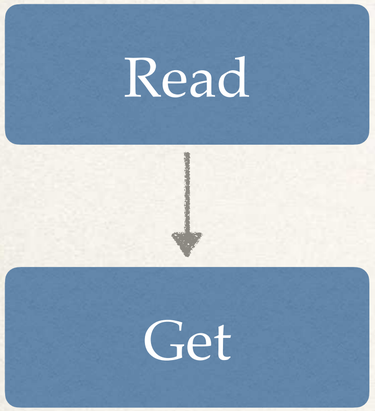
- Path Variables (Path Parameters) are request parameters that have been attached to the URL
- Path Variables are usually defined as a way to find information based on location
- Think of a computer file system:
  - You can identify the specific resources based on the file you are in

```
/Users/luv2code/Documents/java/springboot/section1
```

Path Variables

REQUEST:

URL : localhost:8080/api/books




```
@GetMapping("/api/books")
public List<Book> getBooks() {
    return books;
}
```

Path Variables

```
public void initializeBooks() {
    books.addAll(List.of(
      new Book("Title One", "Author One", "science"),
      new Book("Title Two", "Author Two", "science"),
      new Book("Title Three", "Author Three", "history"),
      new Book("Title Four", "Author Four", "math"),
      new Book("Title Five", "Author Five", "math"),
      new Book("Title Six", "Author Two", "math")
    ));
}
```

- Retrieve a single book by index in ArrayList

GET

`/api/books/{id}`

Retrieve a single book

`/api/books/0`

`/api/books/1`

`/api/books/2`

Known as a "path variable"

Path Variables

```
public void initializeBooks() {
    books.addAll(List.of(
      new Book("Title One", "Author One", "science"),
      new Book("Title Two", "Author Two", "science"),
      new Book("Title Three", "Author Three", "history"),
      new Book("Title Four", "Author Four", "math"),
      new Book("Title Five", "Author Five", "math"),
      new Book("Title Six", "Author Two", "math")
    ));
}
```

Read

Get

REQUEST:

URL : localhost:8080/api/books/2

```
@GetMapping("/api/books/{id}")
public Book getBookByBookIndex(@PathVariable int id) {
     return books[2]
}
```

RESPONSE:

```
{
      "title": "Title Three",
      "author": "Author Three",
      "category": "history"
}
```

Path Variables

REQUEST:

URL : localhost:8080/api/books/jump

Read

Get

```
@GetMapping("/api/books/{word}" )
public String getBookByTitle(@PathVariable String word) {
    return word;
}
```

RESPONSE:

```
{
    "jump"
}
```

Path Variables

REQUEST:

URL : localhost:8080/api/books/jump%20ship%20now

Read

Get

```
@GetMapping("/api/books/{sentence}"
public String getBookByTitle(@PathVariable String sentence) {
    return sentence;
}
```

RESPONSE:

```
{
    "jump ship now"
}
```

Path Variables

```
public void initializeBooks() {
    books.addAll(List.of(
      new Book("Title One", "Author One", "science"),
      new Book("Title Two", "Author Two", "science"),
      new Book("Title Three", "Author Three", "history"),
      new Book("Title Four", "Author Four", "math"),
      new Book("Title Five", "Author Five", "math"),
      new Book("Title Six", "Author Two", "math")
    ));
}
```

- Retrieve a single book by title in ArrayList

GET

`/api/books/{title}`

Retrieve a single book

`/api/books/title%20one`

`/api/books/title%20two`

`/api/books/title%20three`

Known as a "path variable"

Path Variables

```
public void initializeBooks() {
    books.addAll(List.of(
      new Book("Title One", "Author One", "science"),
      new Book("Title Two", "Author Two", "science"),
      new Book("Title Three", "Author Three", "history"),
      new Book("Title Four", "Author Four", "math"),
      new Book("Title Five", "Author Five", "math"),
      new Book("Title Six", "Author Two", "math")
    ));
}
```

Read

Get

REQUEST:

URL : localhost:8080/books/title%20four = title four

```
@GetMapping("/api/books/{title}")
public Book getBookByTitle(@PathVariable String title) {
    for (Book book : books) {
        if (book.getTitle().equalsIgnoreCase(title)) {
            return book;
        }
    }
    return null;
}
```

RESPONSE:

```
{
    "title": "Title Four",
    "author": "Author Four",
    "category": "math"
}
```

Path Variables

```
public void initializeBooks() {
    books.addAll(List.of(
      new Book("Title One", "Author One", "science"),
      new Book("Title Two", "Author Two", "science"),
      new Book("Title Three", "Author Three", "history"),
      new Book("Title Four", "Author Four", "math"),
      new Book("Title Five", "Author Five", "math"),
      new Book("Title Six", "Author Two", "math")
    ));
}
```

Read

Get

REQUEST:

URL : localhost:8080/books/title%20four = title four

```
@GetMapping("/api/books/{title}")
public Book getBookByTitle(@PathVariable String title) {
     return book.stream()
     .filter(book -> book.getTitle().equalsIgnoreCase(title))
     .findFirst()
     .orElse(null);
}
```

RESPONSE:

```
{
      "title": "Title Four",
      "author": "Author Four",
      "category": "math"
}
```

Order Matters with Path Parameters (maybe remove?)

REQUEST:

URL : localhost:8080/api/books/1

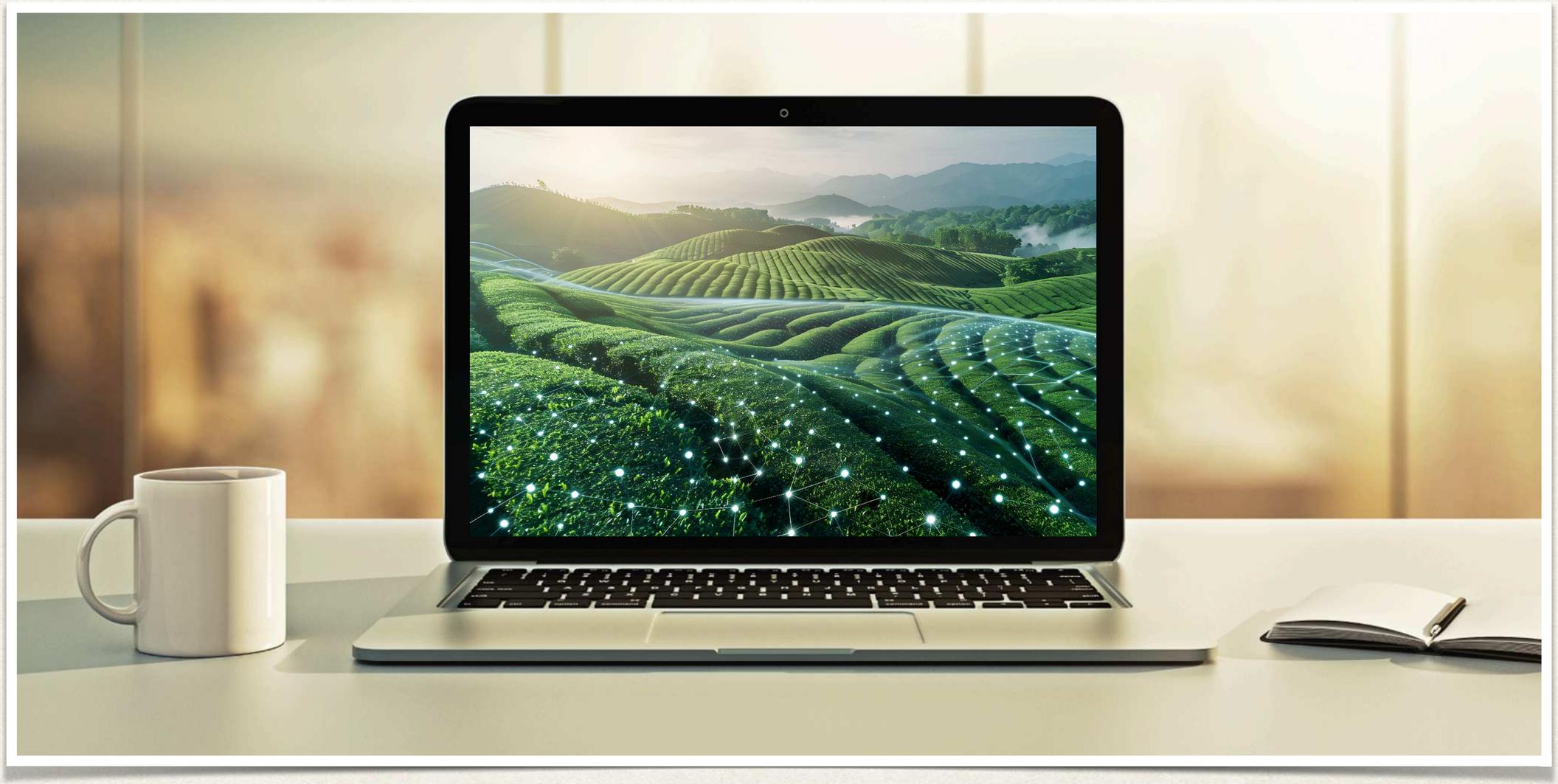
Read

Get

```
@GetMapping("/api/books/{id}")
public Book getBookByBookLis[illegible]cation(@PathVariable int id) {
    return books[2]
}

@GetMapping("/api/books/{title}")
public String getBookByBookTitle(@PathVariable String title) {
    return title;
}
```

URL : localhost:8080/api/books/title%20four

Query Parameters



What are Query Parameters

- Query Parameters are request parameters that have been attached after a “?”.
- Query Parameters have name=value pairs.
- Used for filtering of data.
- Example:
  - localhost:8080/api/books?category=math

Query Parameters

REQUEST:

URL : localhost:8080/api/books?category=science

```
@GetMapping("/api/books")
public List<Book> getBooks(@RequestParam String category) {
    List<Book> filteredBooks = new ArrayList<>();
    for (Book book : books) {
        if (book.getCategory().equalsIgnoreCase(category)) {
            filteredBooks(book);
        }
     }
     return filteredBooks;
}
```

- By default query parameters are required.

Query Parameters

- We can make query params optional.

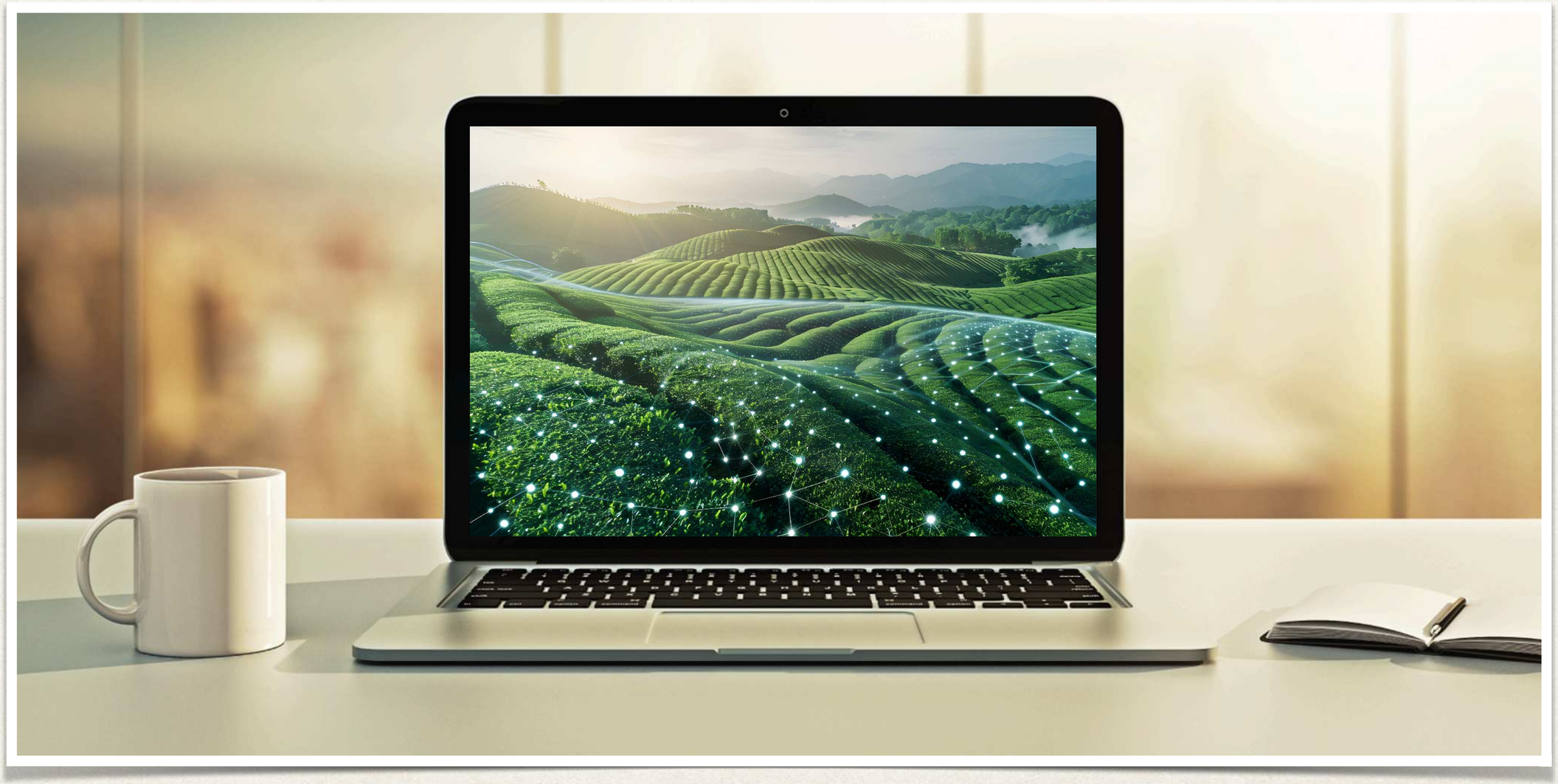
```
@GetMapping("/api/books")
public List<Book> getBooks(@RequestParam(required = false) String category) {

    if (category == null) {
        return books;
    }

    List<Book> filteredBooks = new ArrayList<>();
    for (Book book : books) {
        if (book.getCategory().equalsIgnoreCase(category)) {
            filteredBooks(book);
        }
     }
     return filteredBooks;
}
```

Query Parameters

- Return with a filter stream

```
@GetMapping("/api/books")
public List<Book> getBooks(@RequestParam(required = false) String category) {

    if (category == null) {
        return books;
    }

     return books.stream()
        .filter(book -> book.getCategory().equalsIgnoreCase(category))
        .toList();
}
```

When to use Path vs Query Parameters



When to use Path Parameters

- Identify specific resources.
- Location & Identity is critical.
- /users/{userId}
- /products/{productId}
- /orders/{orderId}
- /articles/{articleId}/comments

When to use Query Parameters

- Identify specific resources.
- Location & Identity is critical.

- /movies?genre=Comedy
- /products?priceMin=10
- /products?priceMin=10&priceMax=100
- /books?category=Science

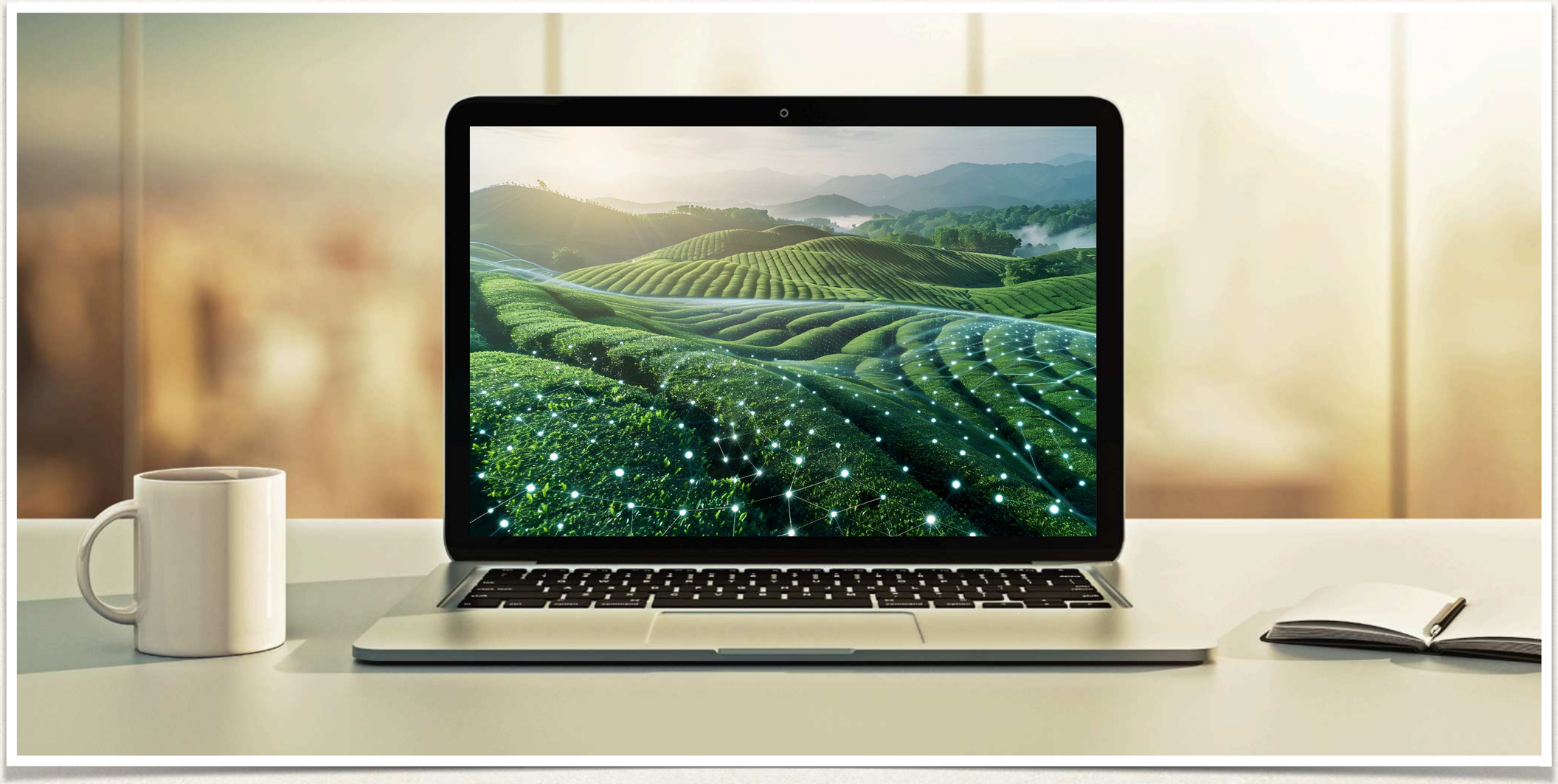
Path + Query Parameters

REQUEST:

URL : localhost:8080/api/books/1?category=math

```
@GetMapping("/books/{libraryId}")
public List<Book> getBooksByLibraryAndCategory(@PathVariable int libraryId,
                                               @RequestParam String category) {
    <fetch books from library>
}
```

POST HTTP Request Method



What is the POST Request Method

- Used to create data
- POST can have a body that has additional information that GET does not have
- Example of Body:

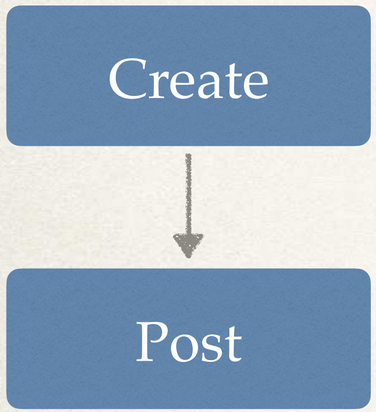
```
{“title”:”Title Seven”, “author”:”Author Two”, “category”: “math”}
```

POST Request Method

```
public void initializeBooks() {
    books.addAll(List.of(
      new Book("Title One", "Author One", "science"),
      new Book("Title Two", "Author Two", "science"),
      new Book("Title Three", "Author Three", "history"),
      new Book("Title Four", "Author Four", "math"),
      new Book("Title Five", "Author Five", "math"),
      new Book("Title Six", "Author Two", "math")
    ));
}
```

REQUEST:

URL : localhost:8080/api/books




```
@PostMapping("/api/books")
public void createBook(@RequestBody Book newBook) {
    for (Book book : books) {
        if (book.getTitle().equalsIgnoreCase(newBook.getTitle())) {
            return;
        }
     }
    books.add(newBook);
```

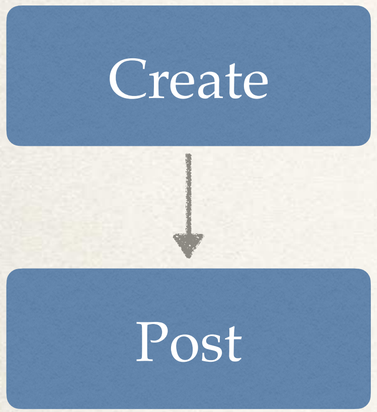
```
{"title":"Title Seven", "author":"Author Two", "category": "math"}
```

POST Request Method

```
public void initializeBooks() {
    books.addAll(List.of(
      new Book("Title One", "Author One", "science"),
      new Book("Title Two", "Author Two", "science"),
      new Book("Title Three", "Author Three", "history"),
      new Book("Title Four", "Author Four", "math"),
      new Book("Title Five", "Author Five", "math"),
      new Book("Title Six", "Author Two", "math")
    ));
}
```

REQUEST:

URL : localhost:8080/api/books




```
@PostMapping("/api/books")
public void createBook(@RequestBody Book newBook) {
    for (Book book : books) {
        if (book.getTitle().equalsIgnoreCase(newBook.getTitle())) {
            return;
        }
     }
    books.add(newBook);
```

```
{"title":"Title Seven", "author":"Author Two", "category": "math"}
```

POST Request Method

```
public void initializeBooks() {
    books.addAll(List.of(
      new Book("Title One", "Author One", "science"),
      new Book("Title Two", "Author Two", "science"),
      new Book("Title Three", "Author Three", "history"),
      new Book("Title Four", "Author Four", "math"),
      new Book("Title Five", "Author Five", "math"),
      new Book("Title Six", "Author Two", "math")
    ));
}
```

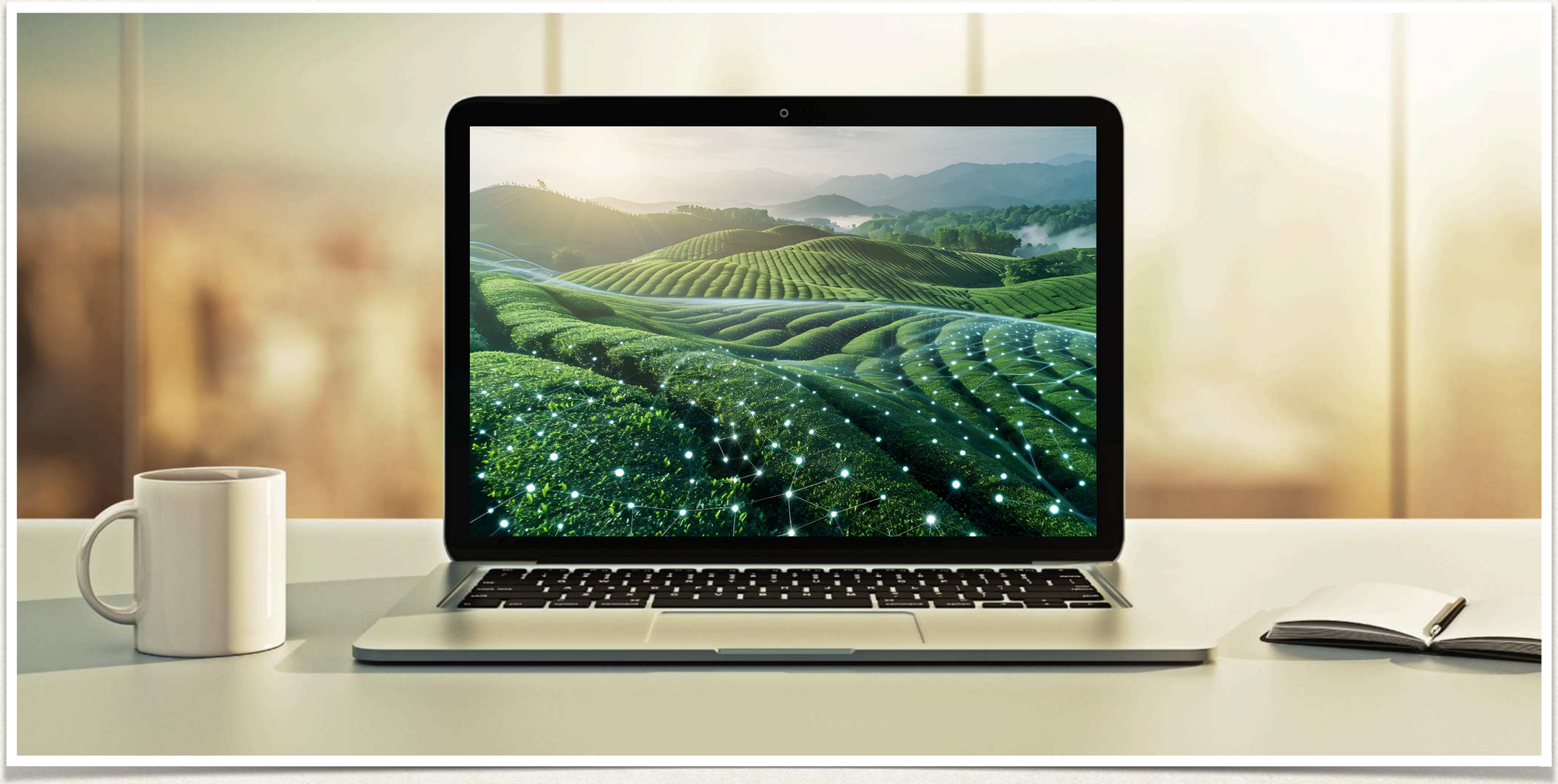
REQUEST:

URL : localhost:8080/api/books

```
@PostMapping("/api/books")
public void createBook(@RequestBody Book newBook) {
    boolean isNewBook = books
      .stream()
      .noneMatch(book -> book.getTitle().equalsIgnoreCase(newBook.getTitle()));

    if (isNewBook) {
       books.add(newBook);
    }
```

```
{"title":"Title Seven", "author":"Author Two", "category": "math"}
```

PUT HTTP Request Method

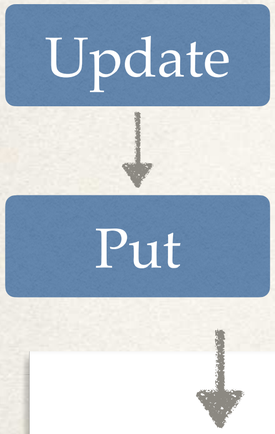


What is the PUT Request Method

- Used to update data
- PUT can have a body that has additional information (like POST) that GET does not have
- Example of Body:

```
{“title”:”Title Six”, “author”:”Author Two”, “category”: “science”}
```

PUT Request Method

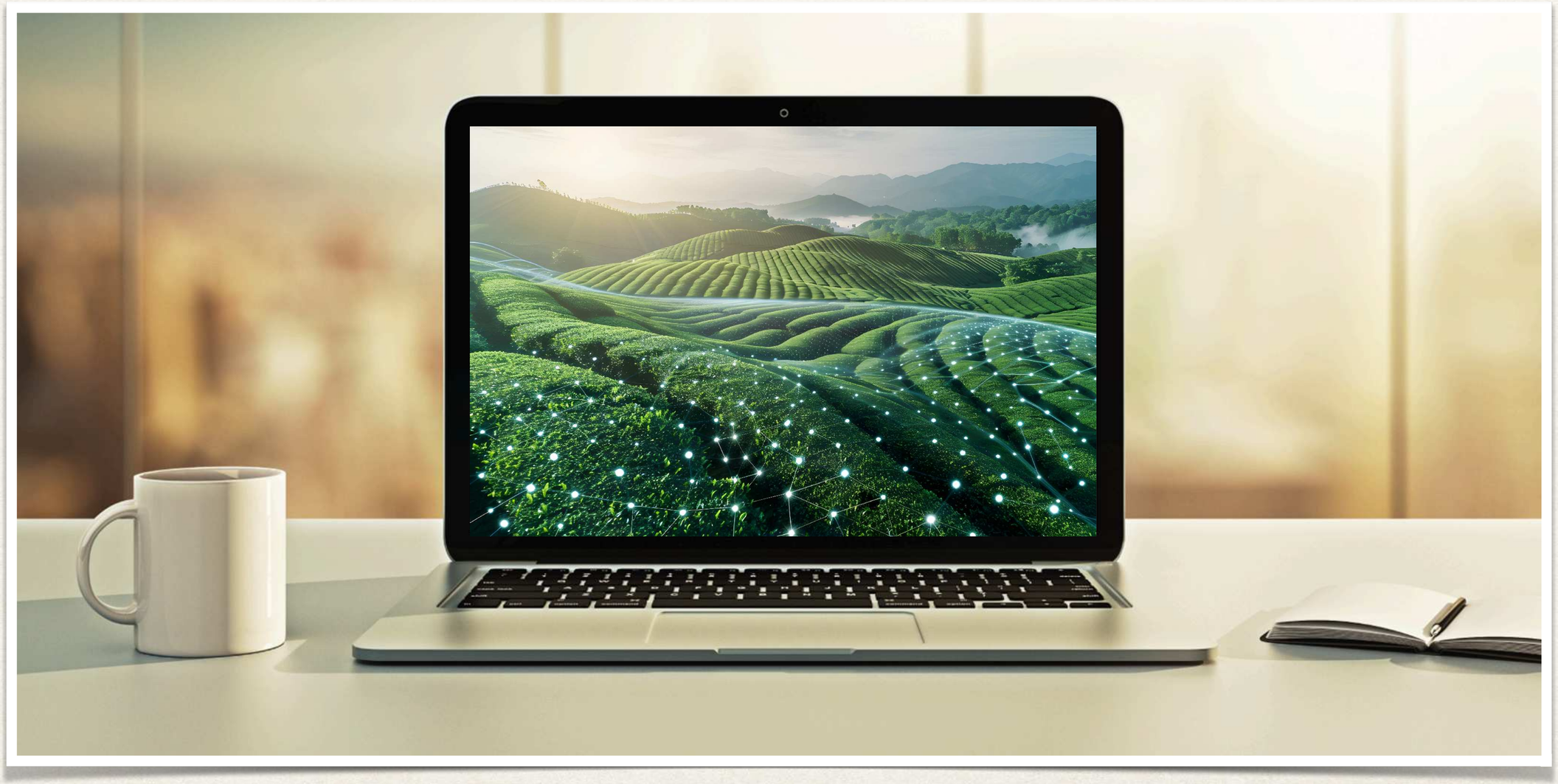


REQUEST:

URL : localhost:8080/api/books/{title}

```
@PutMapping("/api/books/{title}")
public void updateBook(@PathVariable String title, @RequestBody Book updatedBook)
{
    for (int i = 0; i < books.size(); i++) {
        if (book.get(i).getTitle().equalsIgnoreCase(title)) {
            books.set(i, updatedBook);
            return;
        }
    }
}
```

DELETE HTTP Request Method



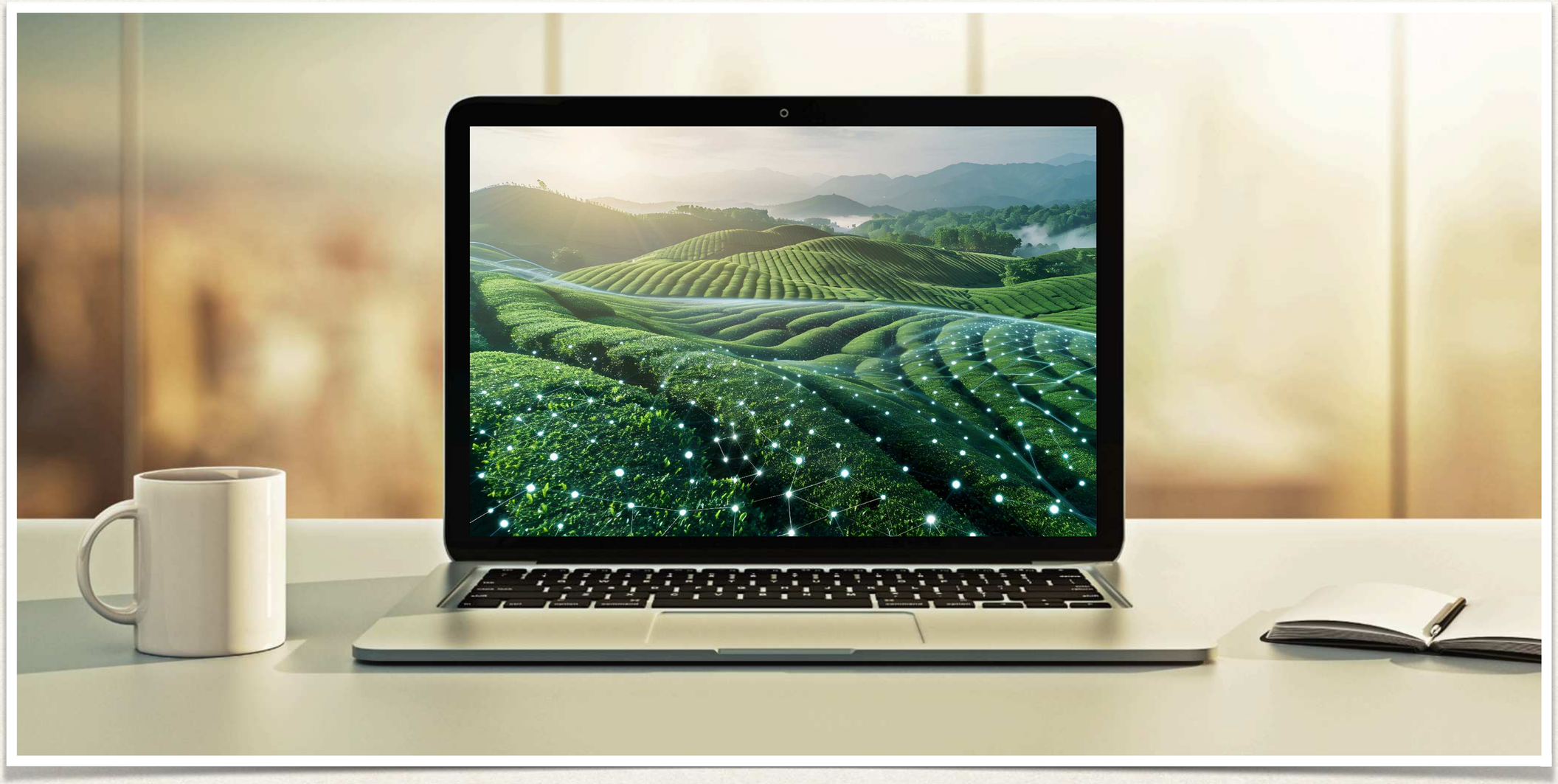
What is the DELETE Request Method

- Used to delete data

REQUEST:
URL : localhost:8080/api/books/{title}

```
@DeleteMapping("/api/books/{title}")
public void deleteBook(@PathVariable String title) {
    books.removeIf(book -> book.getTitle().equalsIgnoreCase(title));
}
```

@RequestMapping



What is @RequestMapping

- So far, we have to add /api/books into each of our endpoints.

```
@GetMapping("/api/books")
@GetMapping("/api/books/{title}")
@PutMapping("/api/books/{title}")
@PostMapping("/api/books")
@DeleteMapping("/api/books/{title}")
```

Each endpoint we need to type in /api/books

What is @RequestMapping

- Add @RequestMapping to a class to auto add init endpoint values.

```
@RestController
@RequestMapping(“/api/books”)
public class Controller {

   @GetMapping
   @GetMapping(“/{title}”)
   @PutMapping(“/{title}”)
   @PostMapping
   @DeleteMapping(“/{title}”
}
```

Keep original code

/api/books is now automatically implemented on each endpoint within this controller